

CCP-D2 Final Report — Naive GPU Implementation Results

Course: CS 4110 - High Performance Computing with GPUs

Team Members: Imaad Fazal, Rayyan Imran, & Saleh Mobashar

Platform: University HPC Server (NVIDIA RTX 3080)

Date: October 14, 2025

GitHub Repository Link: https://github.com/Fastian-afk/HPC_Project

Introduction

This document presents the performance results for Deliverable 2 (D2) of the KLT Feature Tracker, executed on the University HPC system equipped with an NVIDIA RTX 3080 GPU. Following the profiling analysis from Deliverable 1 (D1), the function '_convolveSeparate' was identified as the primary computational hotspot ($\approx 85.94\%$ of runtime). In D2, a naive GPU implementation of this function was developed using CUDA to evaluate baseline GPU performance before introducing optimizations. It is to be noted that as the objective of D1 was only to profile (not to calculate speedup), no custom dataset was made. For D2 a custom dataset of 275 frames (1920 x 1070) was made.

Performance Summary

Table 1. Execution Time Comparison

Version	Execution Time (s)	Speedup Factor	Remarks
CPU Baseline (V1)	1.777	—	Reference CPU implementation
GPU Naive (V2)	3.276	8.94×	Naive CUDA version

Correctness Validation

The GPU output was verified against the CPU baseline to ensure correctness. Both versions produced identical results for all 275 frames of feature tracking. Feature detection and tracking counts matched exactly:

- 1000 features initially detected
- 860 features successfully tracked by frame 275

No discrepancies were observed between CPU and GPU outputs.

GPU Implementation Details

The target for GPU acceleration was '_convolveSeparate', which accounted for the primary computational bottleneck in CPU runtime. The naive GPU version used global memory access with stride alignment optimization, performing separable 2D convolution in two passes : horizontal and vertical. Each CUDA thread processed a single output pixel, with convolution kernel coefficients stored in constant memory.

- Thread block size: 32 x 8 (256 threads per block)
- Grid configuration: dynamically computed based on image resolution
- Memory: global memory with 32-byte stride alignment (warp-friendly access)
- Constant memory: convolution kernel coefficients (up to MAX_KERNEL_WIDTH)
- Kernel launches: two per convolution operation (horizontal and vertical passes)

Performance Analysis

The GPU version achieved a speedup factor of 8.94× over the CPU baseline (29.281s → 3.276s). However, detailed profiling reveals that only 3.15% of GPU time is spent on actual computation (0.103s kernel execution), while 96.9% is consumed by memory transfer overhead (3.173s). The raw computational speedup is 284× (29.278s CPU convolution time → 0.103s GPU kernel time), indicating highly efficient kernel implementation with significant memory bottleneck.

The primary causes limiting overall speedup include:

1. Repeated host-device (H2D and D2H) memory transfers for each convolution call (1098 total transfers across 549 gradient and 549 smoothing operations)
2. No persistent GPU memory — data transferred for every frame operation
3. Absence of shared memory tiling (redundant global memory access)
4. Global memory bandwidth bottlenecks
5. No kernel fusion between horizontal and vertical passes
6. No use of CUDA streams or asynchronous concurrency

Despite memory bottlenecks, the correctness and stability of GPU results confirm a successful baseline port, achieving pixel-perfect matching with CPU reference across 275 frames with 850-1000 tracked features.

V3 Optimization Targets

Based on the D2 results, the next phase (V3) will focus on optimizing GPU performance by addressing current bottlenecks. Planned improvements include:

- Persistent GPU memory allocation - expected 10-20× improvement by eliminating repeated transfers
- Kernel fusion (horizontal + vertical) - expected 30-50ms reduction in kernel time
- Shared memory tiling - expected 3-5× improvement in memory bandwidth utilization
- Texture memory caching - expected 2-3× improvement in spatial locality access
- CUDA streams for overlapping data transfer and computation - expected 1.5-2× improvement

The combined goal for V3 is a total speedup of 50-150× compared to the CPU baseline, targeting 0.2-0.6 seconds total execution time.

Conclusion

The D2 deliverable successfully demonstrates a naive GPU implementation of the key computational hotspot in the KLT pipeline. The implementation achieves 8.94× speedup with a 284× raw computational

acceleration, establishing both correctness and significant optimization potential. The version establishes a correct and verifiable CUDA baseline with pixel-perfect accuracy across 275 frames. Future optimization work in V3 will focus on eliminating memory transfer bottlenecks through persistent GPU allocation, leveraging shared memory, and kernel fusion to achieve significant acceleration over the CPU baseline.